Comparing Automatic Text Summarization with Graph Simplification Algorithms for Explanatory Models

Tyler J. Gandee, Philippe J. Giabbanelli

Department of Computer Science & Software Engineering, Miami University, INSERT NEW ADDRESS, Oxford, 45056, OH, USA

Abstract

Explanatory models like causal loop diagrams and interrelationship digraphs aid users to visually understand complex problems. These models have been used to help understand problem areas like obesity and suicide in youth. Unfortunately, as problem spaces increase in their complexity, it also affects the model's size. Even a textual representation of the explanatory model results in large amounts of text. Although the text is linear, it is still difficult to read with every concept being accounted for. This paper uses a case study in suicide in youth, originally with 361 nodes and 946 edges. We propose methods to compare automatic text summarization (ATS) with graph reduction and text generation methods to form a cohesive summary of the original causal map that contains the maximum amount of information with minimal text displayed. Our graph reduction algorithms include removing singly-linked chained nodes and pruning nodes with a large number of edges. We then calibrate our reduction algorithms until our desired size is met, which will vary depending on the causal map's original size and desired reduced graph. We also describe our algorithm for processing large amounts of texts to summarization models in batches. We conclude by comparing summaries through automatic and manual evaluations. We utilize extractive summarization methods TextRank, BERT, and Longformer, as well as abstractive models BART, T5, Longformer-Encoder-Decoder (LED). We also use GPT-4 as a comparison for Large Language Models (LLMs). We found that while we did achieve the best scores with BART, when dealing with various ATS models, it is generally advisable to use generative AI as little as possible to minimize risks of hallucinations. Our graph reduction method does this by removing the text summarization step entirely, and thus providing a consistent and reliable summary at the cost of worse verbiage.

Keywords: Natural Language Processing, Automatic Text Summarization

1. Introduction

Explanatory modeling aims to establish causal relationships within a set problem domain [1]. They primarily differ from computational models in that explanatory models are strictly for visual representation. However, it is possible to extract an explanatory model from a computational model, e.g. extracting a causal map from a Fuzzy Cognitive

Map (FCM) [2, 3]. While computational models provide a simulation of an environment, explanatory models remain the most accessible to understand for various users, including Subject Matter Experts (SMEs), policymakers, and community members [4].

Causal models are an explanatory model that explains problems using cause-and-effect relationships. Each edge demonstrates either a positive or negative causality, describing the impact on the target node given the source node becoming more apparent.

One of the main problems with causal modeling is its scalability. As causal models are used in more complex domains, this complimentary increase's it's causal map size to ensure the problem is understood from various perspectives [5]. Eventually, the causal map becomes so large that is resembles a "spilled plate of spaghetti" more than a causal map [6, 7, 8].

This problem is analogous to large amount of texts. There is extensive research on reducing the amount of text one must read through automatic text summarization methods [9]. There are also efforts to reduce the size of graphs [10]. In both cases, the goal is to maximize information while minimizing redundancy [11].

In this paper, we propose methods to identify the best summarization algorithm for causal maps. We provide a case study of suicide in youth to demonstrate how to identify the best method. In particular, since we have the causal map and a textual representation of that map, we are comparing automatic text summarization methods through Large Language Models (LLMs) with graph reduction algorithms. Our reduced graph is then transformed into text as the textual version was.

The remainder of this paper is organized as follows. In Section 2, we provide a brief background to generating text of causal maps, as well as automatic text summarization and graph reduction methods. In Section 3, we describe the algorithms used to apply our summarization models and reduce our graph to a desired size. Our results are displayed in Section 4, and we compare all generated sumamries with handwritten summaries of the causal map. Lastly, we discuss our findings in Section 5 and provide concluding remarks in Section 6.

2. Background

Causal models, or causal loop diagrams (CLDs) are a form of explanatory models that are represented as directed networks [4]. The provide cause-and-effect relationship to demonstrate how a change in one concept may result in changes of other concepts.

Causal models can be compared to its closest equivalent in simulation models, the Fuzzy Cognitive Map (FCM) [3]. While causal models are solely explanatory (through graph visualizations) with only positive and negative effects, FCMs can be sun to simulate system dynamics, with weights ranging from $[-1, 1]$ [12]. Each named concept then provides non-linearity given an activation function e.g. sigmoid or tanh. Given its architecture, FCMs have also been defined as recurrent artificial networks [13].

Given that FCMs are both runnable and can be represented visually, it provides a strong medium between simulation and explanatory modelling. FCMs can be converted to causal models, but causal maps cannot be converted to FCMs without further calibration. To convert an FCM to a causal map, all negative edges are mapped to $-$, and positive edges

to +. If any edges are 0 or neutral, they may optionally be removed, since this would not impact the FCM or causal map's outcomes. Activation levels are removed thus only leaving the concept name. Hester *et al.* provide an example with the Ebola Virus, with the model beginning as a causal map and later expanded to an FCM [14, 12].

While FCMs and causal maps share similar notions, they also share similar drawbacks. First, both models struggle with scalability. The more complex a domain is, the larger the model becomes [15, 16]. For example, in the case of *suicide in youth*, the causal map contains 361 nodes with 946 interrelationships [5]. With the amount of information present, the model becomes unorganized and resembles a "spilled plate of spaghetti" [17, 7]. In the case of FCMs, increasing model complexity also means increasing simulation runtime.

Since explanatory models are *explainable* at the human level, a textual representation of the model can be created [18, 19]. In particular, the causal map is decomposed in a linear way to avoid redundancy from feedback loops [4, 19]. While we can use textual representations as another medium to explain causal relationships, we have simply transferred the issue from being a complex graph to large text.

Due to the amount of text growing daily, there has been a demand for Automatic Text Summarization (ATS) models and algorithms. There are two main summarization methods: abstractive and extractive.

Abstractive summarizations aim to resemble human-like summaries by creating new sentences based on the original text [20]. Many abstractive summarization models are created through transfer learning, by pre-training a LLM and fine-tuning it for summarization tasks [21, 22]. This allows tokens outside of the original text to be used to create a cohesive summary than sorting sentences by significance.

Contrarily, that process is extractive summarization. Instead of generating new sentences, the existing sentences are used to highlight the most-significant pieces of text. While the summary itself may feel disconnected between sentences, outputting only existing sentences leaves no room for possible hallucinations, which is a common problem in language models.

Figure 1 gives a brief example visualizing the token process of abstractive and extractive models. Abstractive summarizations work similar to generative models, by creating the summary based on the previously-generated words. Extractive models are similar to binary classifiers, as each sentence is decided the model to be included or excluded from the summary.

There are other methods to extractive and abstractive summarizations besides transfer learning. TextRank is a graph-based algorithm that sorts sentences from most-to-least significance, allowing the summary to be deterministic when generating extractive summaries [23]. Sequence-to-sequence recurrent neural networks (RNNs) and pointer-generator networks have also been used for abstractive summarization [24, 25].

3. Methods

3.1. Overview

Our methods are divided into two main sections (Figure 2). First, we generate paragraphs that fully explain a causal map. This gives us a graph-based and text-based variation of the

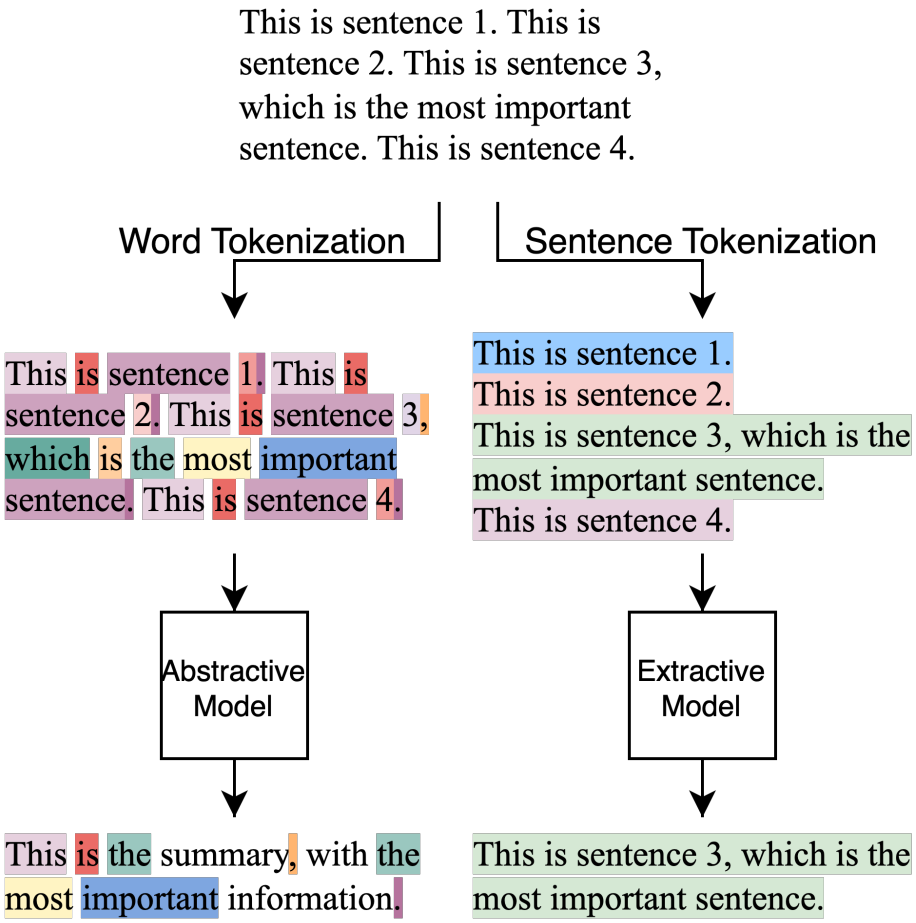


Figure 1: Example to understand the pre-processing differences in extractive and abstractive summarizations.

causal map. We use the text-based form for ATS, and the graph-based form for graph simplification.

To simplify the graph, we first remove all self-loops. We then apply our pruning algorithm. The first two steps focus on removing *edges*. To remove nodes, we apply our chain compression and trimming algorithms until there are no more nodes to remove. The process of text summarization is slightly simpler. We take the entire map and immediately generate paragraphs. Then, given the generated paragraphs, we use this text to run each fine-tuned model, which consists of TextRank [23], BERT [26], Longformer [27], BART [28], T5 [22], Longformer Encoder-Decoder (LED) [27], and GPT-4 Turbo. Finally, we evaluate given the generated summary.

3.2. Graph Simplification

For graph simplification, we developed 3 algorithms to aid in removing as many nodes and edges as possible, without removing key factors e.g. ACEs and Suicide ideation. We provide a dispatch algorithm for the sole purpose of how we utilize all the algorithms to reach

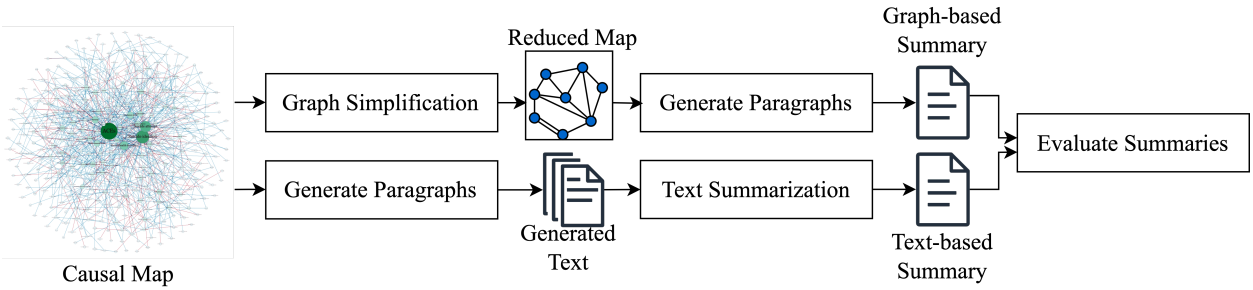


Figure 2: Main steps layed out in this paper to compare graph-based and text-based summaries.

our desired graphs size. In Algorithm 1, we compress all chain-like sequences of edges, i.e. we remove intermediates between nodes until we find branches or loops. Figure 3 provides an example of how trimming and chain compression can reduce the size of our graphs in one iteration. In Algorithm 2, we prune the least-central nodes until the maximum number of edges n is met. For our case, we found we achieved a satisfactory length with our maximum edges set to 2, along with determining least-central nodes based on their lowest betweenness.

The process ends once there are no more nodes to remove, and we can determine the map has been simplified. Lastly, we generate a new set of paragraphs using our generative model, and evaluate the graph-based summary. Algorithm 3 shows how we pre-process our graph (e.g. removing self-loops and trimming source/sink nodes), utilize our defined algorithms, and tune all parameters to achieve our desired graph size.

In pre-processing, we remove self-loops, then prune our graph until we are satisfied with the maximum number of outgoing edges. Then, we apply chain compression and trimming across the entire graph, and repeat chain compression and trim until there are no more changes between iterations. The result is our simplified map.

3.3. Automatic Text Summarization

Of our 6 ATS processes, 5 utilize a machine-learning approach of training a model to generate a summarization. The only exception is TextRank, which uses a graph-based approach to rank the existing sentences by significance. The parameter for each model is the ratio of the summary length in comparison to the text length, e.g. a ratio of 0.2 would yield a summary that is 20% the length of the original text.

To identify the ratio for evaluation, we use a ground-truth summary generated from our graph simplification methods. This ensures all models and methods are comparable by having a roughly-fixed length. This summary resulted in our ratio being 0.114. Model details, including names and libraries for implementation, can be found in Table 1.

To ensure the preservation of sentences through extractive models and NLP techniques, the *sentences* are tokenized and passed through the model, and clustered using K-Means [26]. This is how sentences are preserved through similar transformer methods, to allow both NLP and NLG tasks. However, abstractive summarization tokenizes *words* to generate new sentences, so the BERT-extractive-summarizer cannot be used here. Therefore, there are significantly more tokens needed to process abstractive summarizations than extractive,

Algorithm 1: Chain Compression in a Graph

```
Input: Graph G, node to check for compression, set of nodes to preserve void
Output: Set of nodes to potentially remove, Boolean indicating if compression or
        voiding occurred
Function ChainCompression(G, node, void)
    if void is None or node ∉ void then
        in_edges ← G.in_edges(node, with data=True)
        out_edges ← G.out_edges(node, with data=True)
        head ← None
        weight ← 1
        /* weight keeps track of the edge property between the head and
           tail of the chain.                                            */
        temp ← {node}
        while len(in_edges) = 1 do
            if head ∈ temp then
                return temp \ {None}, False
                /* in_edge loop                                          */
            temp ← temp ⊎ head
            head ← in_edges[0][0]
            weight ← weight× int(in_edges[0][2]['weight'])
            in_edges ← G.in_edges(head, with data=True)
        if head ≠ None then
            tail ← None
            in_edges ← {None}
            while len(out_edges) = 1 and len(in_edges) = 1 do
                if tail = head then
                    return temp \ {None}, False
                    /* out_edge loop                                     */
                temp ⊎ tail
                tail ← out_edges[0][1]
                weight ← weight× int(out_edges[0][2]['weight'])
                out_edges ← G.out_edges(tail, with data=True)
                in_edges ← G.in_edges(tail)
            if tail ≠ None and tail ≠ head then
                G.add_weighted_edge(head, tail, weight)
                return temp \ {None}, True
                /* Head and tail are different, so chain exists.         */
    return None, None
    /* No change.                                                        */
```

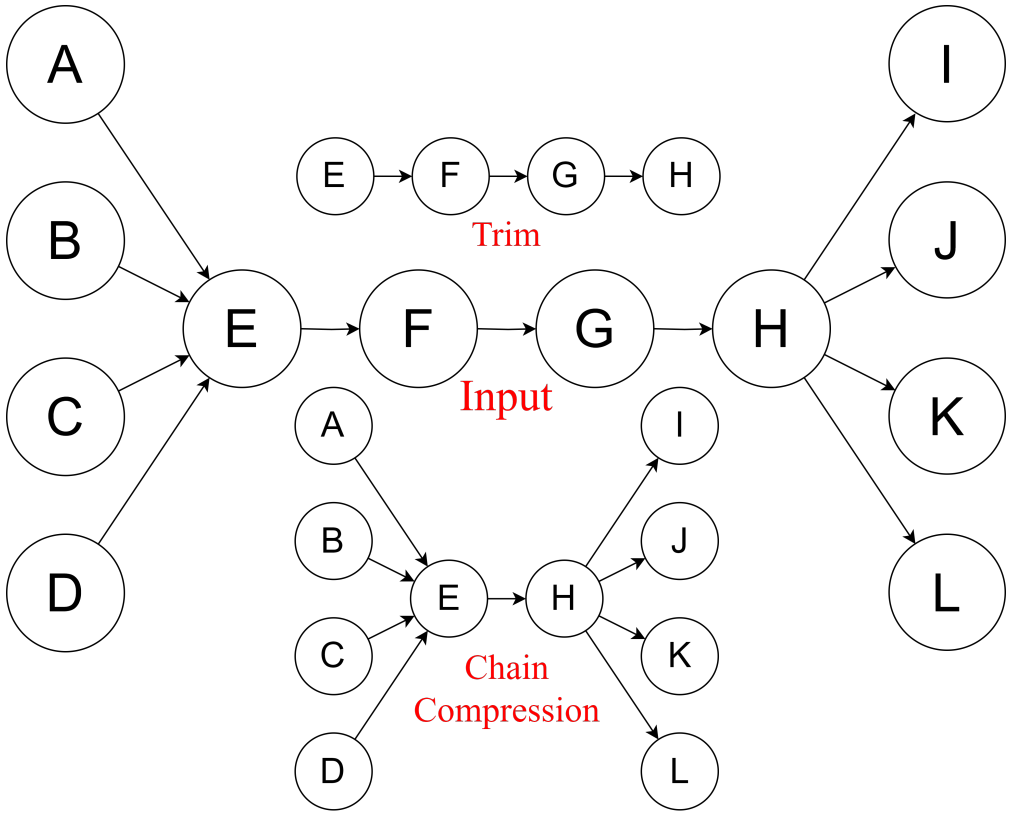


Figure 3: Examples of chain compression and trim using a causal map with 12 nodes and 11 edges after 1 iteration.

which becomes an issue with models having token limits [35], as demonstrated in Figure 1. Thus, we utilize two algorithms to handle pre-processing of the input, running of the model, and post-processing of the generated summary. Algorithm 4 defines how we split our text into batches to fit the model's token length. Algorithm 5 executes Algorithm 4, refines the output tokens, and concatenates each batch's summary into the final summary. These algorithms presented represent one of many techniques to handle token limitations [35, 36, 37].

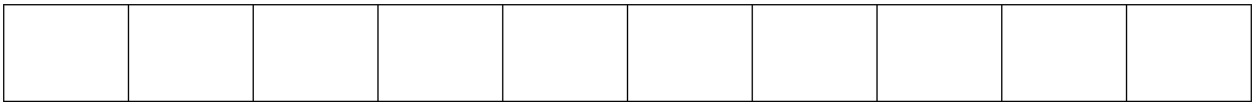
For each summarization model, there is also a corresponding tokenizer. Thus, the number of tokens for any given input text will vary depending on the model. For evaluation, using GPT's tokenizer, our input text consists of $9,540$ tokens.

The goal of Algorithm 4 is to split a long text into smaller batches of similar sizes. Here, instead of using the size of the text, we use the number of tokens generated by the text, since this is what we pass to each model. Once we have the total number of tokens, we divide it by the maximum tokens the model allows and round up. This gives us the number of batches we need to create, while also being under the maximum token limit for each batch. As we build each batch, we add paragraphs to find the difference of the current tokens and the expected tokens. We allow some room for our batch tokens to exceed the expected size by rounding our number of batches up. For example, let's assume our model takes 512 tokens, we expect 400 tokens per paragraph, but we have 2 paragraphs, one with 500 tokens, and

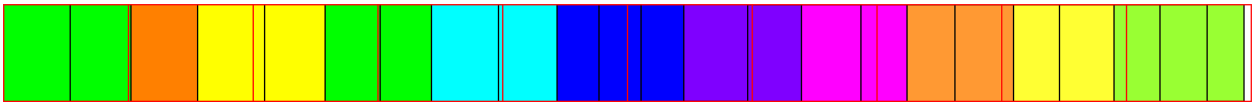
Algorithm 2: Remove the least-central nodes for node with a large amount of outgoing edges

Input: node, node's edge subgraph, max edges, centrality function f
Output: Set nodes to remove as edges from the original node

Function `Prune(`*node, node's edge subgraph, max edges, centrality function, graph centralities*`)`
  `/* Map each subgraph node to its centrality */`
  $centralityMap \leftarrow \emptyset$
  for $node \in subgraph$ **do**
    $centralityMap[node] \leftarrow f(node, G)$
  Sort nodes in $centralityMap$ by value
  If max edges $= 0$, **return** all nodes in *centralities*
  Else **return** all but the max edges nodes in *centralities*



(a) Start Batch (with expected batch sizes)



(b) End Batch

Figure 4: Initialization and result of our batch algorithm using BART, using colors to identify unique batches. The maximum length for BART is 1024 tokens. For a full animation, see https://osf.io/98kmy/.

another with 300 tokens. The first paragraph exceeds the expected number of tokens, but we are still within our token limit. Once the difference begins to increase or our presented batch exceeds the token limit, we do not add the presented paragraph to the current batch, and instead append it to the start of the next batch. Figure 4 provides a visual representation of the beginning and end of our batch algorithm, but using black lines as our expected batch size, with colors indicating the actual size of each.

Algorithm 5 utilizes algorithm 4 to pass all input text through the summarization model regardless of text size. First, the text is batched, and the summarization model is initialized. To compare with graph simplification methods, we provide an optional ground-truth summary (*gt_summary*). If one is provided, we set the ratio and number of paragraphs to be equivalent to the ground-truth summary. In our case, our ideal summary generated from graph simplification (with set parameters as discussed in Section 3.2) yielded nearly 11.36% length of the original text within 3 paragraphs. Once the ratio and paragraph count is set, we iterate through each non-empty batch. For each batch, we get the maximum length of the summary, which is 5% larger than the batch's word limit. Adding some extra space allows us to clean up the text for the output to be closer to the desired ratio. Then, we generate

Algorithm 3: Process Graph

```
Input: Graph G
Output: Simplified Graph G
Function Process(G)
    initialNodes ← |G.nodes|
    initialEdges ← |G.edges|
    oldNodes ← initialNodes
    oldEdges ← initialEdges
    newNodes ← −1
    newEdges ← −1
    i ← 0
    Remove self-loops from G
    /* Prune nodes based on betweenness centrality                    */
    for node in G.nodes do
        for target in Prune(node, 2, nx.betweenness_centrality) do
            /* For our use case, we found that a limit of 2 edges and
               betweenness centrality sufficed.                        */
            G.remove_edge(node, target)
    Remove nodes with degree 0 from G
    while oldNodes ≠ newNodes or oldEdges ≠ newEdges do
        if i > 0 then
            oldNodes ← newNodes
            oldEdges ← newEdges
        void ← ∅
        removeNodes ← ∅
        for node in G.nodes do
            cc ← ChainCompression(G, node, void)
            if cc[1] then
                /* We have identified a chain to remove.              */
                removeNodes ← removeNodes ∪ cc[0]
            else if cc[1] is False then
                /* We have found nodes and edges to preserve.         */
                void ← void ∪ cc[0]
            removeNodes ← removeNodes∪ Trim(node)
            /* trim removes connected source and sink nodes.          */
        G.remove_nodes_from(removeNodes)
        newNodes ← |G.nodes|
        newEdges ← |G.edges|
        i ← i + 1
        Remove nodes with degree 0 from G
    return G
```

Table 1: All models with their corresponding fine-tuned model, along with their token limits and python libraries used for implementation.

Type	Python Library	Fine-tuned Model Name	Token Limit	Model
Extractive	gensim 3.8.2 [29]	N/A	N/A	TextRank [23]
	bert-extractive-summarizer 0.10.1 [26]	bert-base-uncased [30, 31]	512	BERT
		allenai/longformer-base-4096 [27, 32]	$4,096$	Longformer-Ext
Abstractive	transformers 4.35.2 [33]	sshleifer/distilbart-cnn-12-6 [28]	$1,024$	BART
		t5-base [22]	512	T5
		patrickvonplaten/led-large-16384-pubmed [34]	$16,384^*$	LED-base-book-summary
	openai 1.3.3	gpt-4-1106-preview	$128,000^*$	GPT-4 Turbo

*This token limit is larger than our text hence we do not need to split our data into batches when using these models.

the batch summary with *summarizer* pipeline, and clean up the generated summary by removing unfinished sentences, reformatting punctuation, and capitalizing the first letter of sentences. Various models have different pre-processing and post-processing techniques in encoding/decoding, which can result in poorly-formatted text. For example, t5-base does not capitalize the first letter of a sentence (can be seen in [38]). Our observations for each model are defined in Table 2. Finally, we have to append our sentences to either a new or existing paragraph, which adheres to $gt_{paragraphs}$. We do this by finding the modulo of the current batch and the expected number of summaries per paragraph. If we need a new paragraph, we add '\n \n'. If not, we add one extra space for the next summary.

As mentioned in Table 1, different models have varying token limits, which differentiates their output of Algorithm 4. BART was split into into 11 batches, and T5 was split into 24 batches. However, LED and GPT-4 Turbo are a different story. Since out input text does not exceed these models token limits, we can pass them through one large batch[1].

[1]When using LLMs, without model parameters to force the number of generate tokens, it can be difficult to control the length of the output generated. LLMs do not perform particularly well when prompted to stay within a token limit, as was the case for GPT. We can set a maximum number of tokens, but we consistently fell short of our desired length unless a higher temperature resulted in infinite noise. We attempted to mitigate this issue by batching our input as we do for all other summarization models. However, this is unfortunate considering GPT's input token size is significantly larger than other models used.

Algorithm 4: Split text into batches

```
Input: text, model name, model tokenizer, model's max tokens
Output: list of paragraphs that can be batched together
Function Batch(text, name, tokenizer, max tokens)
    paragraphs ← text split by '\n'
    batches ← ∅
    total_tokens ← tokenizer(text)
    batch_length ← ⌈total tokens / max tokens⌉
    div ← total_tokens / batch length  /* div is the expected number of tokens per
        paragraph.                                                              */
    i ← 0
    previous_difference ← tokens
    difference ← −1
    for paragraph ∈ paragraphs do
        if paragraph ≠ ∅ then
            temp ← batch[i] + paragraph
            tokens ← tokenizer(temp)
            Get difference between tokens and div
            if diff_prev < diff or tokens > max tokens then
                i ← i + 1
                batch[i] ← paragraph
                previous_difference ← tokens
            else
                batches[i] ← temp
                diff_prev ← diff
    return batches
```

3.4. Evaluation

For evaluation, we compared all generated summaries with handwritten summaries. We requested the summaries to be 1-2 pages in length with 3 paragraphs to match the length of the generated outputs. One summary focuses on summarizing the causal map from a socio-ecological perspective, while another summary describes factors relating to Adverse Childhood Experiences (ACEs) and suicide ideation. We use these reference summaries as our expected summary for ROUGE. We also provide manual evaluations for faithfulness and fluency on a 5-point Likert scale, with 1 being the worst and 5 being the best [19, 39, 40]. Faithfulness focuses on measuring how few hallucinations exist from GPT (i.e., a score of 1 would be an abundance of hallucinations, while 5 would be no hallucinations). Fluency refers to readability of the text.

Table 2: Comparison of textual formatting across abstractive summarization models.

Model/Criteria	Capitalization	Punctuation	Space	Parentheses	Space	Line Breaks
BART	✓	✓		✓		✓
T5		✓		✓		✓
LED		✓				

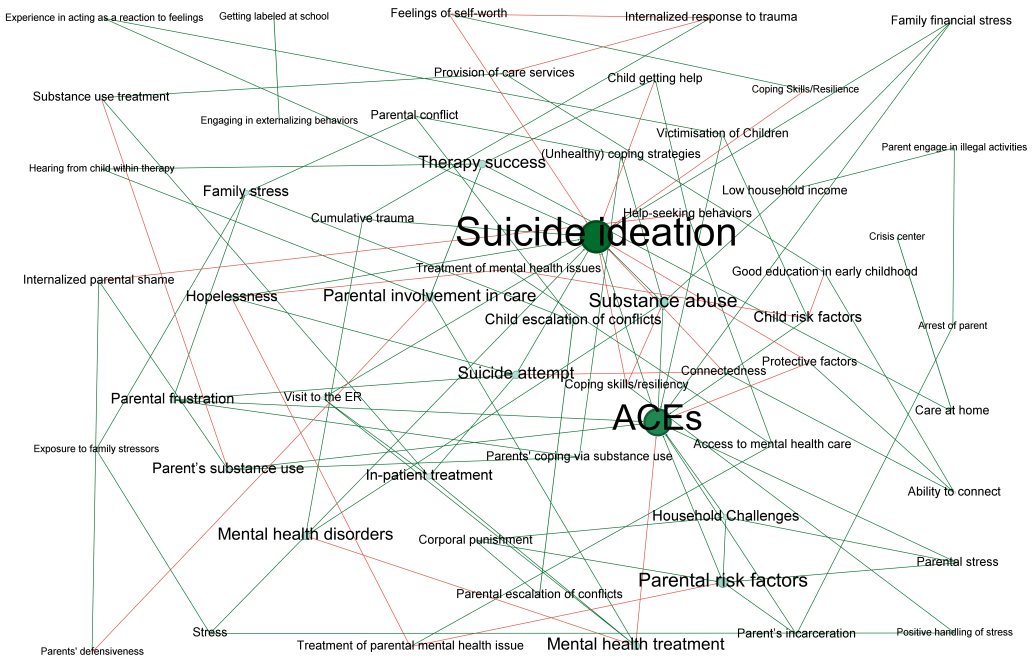


Figure 5: High-definition illustration of the simplified map.

4. Results

4.1. Graph Reduction

As described in Section 3.2, we set the maximum number of edges to 2 for our pruning algorithm. With this, Our graph reduction method yielded a causal map containing 57 nodes with 103 edges, leading to an 84% decrease in nodes and 89% decrease in edges (Figure 5). When we transform this causal map to text (as we have the original graph), we achieve a summary of 945 words and 5979 characters. This is then considered our target length for automatic text summarization to compare relatively equal lengths of texts.

4.2. Evaluation

Our results are shown in table 3, with the highest scores being bold. BART achieved the most number of high scores, including ROUGE-1, ROUGE-2, and ROUGE-LSum, and BERTScore recall, while textRANK achieved the highest Rouge-L score. Longformer-EXT achieved the same ROUGE-2 score as BART and outperformed on METEOR and BLEU,

Algorithm 5: Generate abstract

Input: text, model name, model tokenizer, model's max tokens, optional ground truth summary

Output: summary paragraphs generated from text

```
Function GenerateAbstract(text, name, tokenizer, max tokens, gt_summary)
    batches ← batch(text, name, tokenizer, max_tokens)
    summarizer ← pipeline(name)
    if gt summary ≠ ∅ then
        r ← word count(gt summary) / word count(text)
        gt_paragraphs ← paragraph count(gt_summary)
    else
        r ← 0.1
        gt_paragraphs ← 3
    data ← ∅
    for i in range(len(batches)) do
        if len(batches[i]) > 0 then
            max_summary_length ← word count(batches[i]) * (r + 0.05)
            sentences ← summarizer(batches[i],
            max_summary_length)
            Refine grammar of sentences
            if i mod len(batches) / gt_paragraphs = 0 then
                sentences ← sentences + '\n\n'
            else
                sentences ← sentences + ' '
            entry ← {'original_text' : batches[i], 'summary' : sentences}
            data ← data ⊎ entry /* append entry                    */
    return data
```

while LED-base-book-summary achieved the highest BERTScore precision. All models except gpt-4-turbo achieved a BERTScore F1 of 0.84.

For manual evaluation, we found extractive summaries were the most faithful albeit the least fluent. This was expected, as extractive summaries only utilize sentences from their original text rather than generating new sentences. The lack of fluency stems from there being no connection to sentences unless neighboring sentences were selected for the summary. For abstractive summarizations, we observed the opposite behavior. While abstractive summarizations were deemed more fluent, there were some cases of hallucinations through nonexistent concepts and quotes from public health officials. LED-base-book-summary had the highest level of hallucinations, which injected the following quote: "There are many factors that can affect how connected individuals feel in their families and communities." The summary also injected non-existent nodes like e.g. standardized testing and electroshock therapy. There were also minor hallucinations in T5. For example, our summary referenced

Table 3: Evaluation metrics for summarization models in relation to handwritten summaries with ROUGE-1, ROUGE-2, ROUGE-L, ROUGE-LSum, METEOR, BLEU, and BERTscore for precision, recall, and F1 score.

Method	**Model**	**R-1**	**R-2**	**R-L**	**R-LSum**	**MET**	**BLEU**	*Prec*	*Rec*	*F1*
Ext.	textRANK	0.43	0.10	**0.16**	0.32	0.30	0.08	0.85	0.83	**0.84**
	bertEXT	0.45	0.09	0.15	0.22	0.29	0.06	0.85	0.83	**0.84**
	longformerEXT	0.45	**0.11**	0.15	0.29	**0.31**	**0.10**	0.85	0.83	**0.84**
Abs.	distilbart	**0.49**	**0.11**	0.14	**0.33**	0.30	0.07	0.84	**0.84**	**0.84**
	T5	0.45	0.10	0.14	0.26	0.29	0.08	0.84	0.83	**0.84**
	LED	0.14	0.05	0.08	0.10	0.05	0.00	**0.87**	0.81	**0.84**
	GPT-4-Turbo	0.42	0.08	0.14	0.29	0.32	0.05	0.83	0.83	0.83
Graph		0.41	0.09	0.15	0.28	0.28	0.07	0.85	0.83	**0.84**

Table 4: Evaluation metrics on readability and manual evaluations.

Method	Model	FRE	WPS	Lex. Ct.	Faith.	Flu.
	Original Text	53.61	17.56	8322	5	2
	Reference 1	51.07	20.05	782	5	5
	Reference 2	27.22	18.64	783	5	4
Ext.	textRANK	31.82	22.37	917	**5**	3
	bertEXT	54.42	16.76	**922**[*]	**5**	3
	longformerEXT	53.41	**17.84**[*]	981	**5**	3
Abs.	distilbart	53.61	17.56	981	**5**	**5**
	T5	**56.86**	14.35	890	4	4
	LED-base-book-summary	38.55	24.05	457	2	**5**
	GPT-4-Turbo-Preview	7.96	20.90	836	**5**	4
Graph		52.70	18.53	945	**5**	**5**

[*]These values are preferred because they are the closest to our textual representation of the reduced graph.

Simon Tisdall, who is a foreign affairs commentator [41], and is not mentioned in the original text.

Distilbart and GPT-4-Turbo-Preview stemmed to be the most faithful in abstractive summarizations, but Distilbart was more readable as demonstrated by comparing their Flesch Reading Ease score and words per sentence. GPT-4-Turbo-Preview is readable, but the model contained more abstract verbiage using words e.g. "exacerbate", "strengthening" and "community-focused".

5. Discussion

While our graph reduction method stands comparable to ATS models, it underperforms as a total replacement, only being able to achieve the same F1 BERTScore as all other models except for GPT-4-Turbo. BART (sshleifer/distilbart-cnn-12-6) is the clear choice by outperforming 3 of the 4 ROUGE scores along with Recall and F1 BERTScore. However, we must recall that the abstractive summarization models are designed to be trained to generate a summary given some text (excluding GPT-4-Turbo which was only given a role). When converting the reduced graph to paragraphs, the generative model does not know it is processing a reduced graph. Thus, the summary is limited in the information it *could* use because it depends on the nodes and edges removed in the simplification process. For example, if there are 27 edges connected to ACEs and we prune all but 2 edges, our graph reduction method means the model believes there are 2 *and only* 2 edges, thus affecting the amount of knowledge retained. Furthermore, both abstractive and extractive summaries have sentences verbatim from the original text (this is expected with extractive summarization). When we reduce the graph, we are creating a new list of subgraphs, which then generates a new list of sentences and paragraphs that differentiates from our original graph, and thus original text. For this reason, it is understandable that similarity metrics may not be satisfactory because it is considering subgraphs that have not been directly observed by the writer of the handwritten summaries.

Utilizing generative AI in causal modeling is still relatively new, but has been researched in other areas e.g. converting knowledge graphs to text [42]. There is also extensive work in graph reduction and text summarization, but never in-tandem with one another as we have demonstrated in this paper. Therefore, this field would highly benefit from a benchmark dataset with data containing a causal map, textual representation of the map, and summarized versions of the text and graph. This would also allow us to fine-tune our own summarization model and develop a graph reduction algorithm that is generalizable rather than developing methods for a specific case study.

Writing a summary for an original text can be conflicting, especially when all other automated summaries are aiming for a target length. Our graph reduction text provided a deterministic causal map that would be transformed into text, giving us a target length to aim for in other summary methods. Although, our reference summaries are around 17% shorter than the graph-based summary. We chose to focus on the graph's summary because the reduction method applies to the entire graph until there is no change from the previous iteration (per Algorithm 3) rather than until the graph is a set size, because it would be

unknown which nodes and edges are removed given the same conditions. For example, if we only want a node to be connected to 2 edges but we have already met our desired number of edges, some nodes may have 3 or more edges, and there is no way to tell which edges are preserved before the algorithms are ran. Ideally, the handwritten summaries and reduced graphs would contain the same information, but this is not present in our case.

6. Conclusion

In this work, we have demonstrated a new solution to further understanding large causal maps by reducing their size both structurally and textually. We have demonstrated that our proposed algorithms for reducing the size of causal maps is comparable to other ATS methods, with faithfulness and fluency achieving the same scores as BART. While our graph reduction methods did not achieve optimal scores compared to some evaluation metrics, we noticed an improvement in faithfulness and fluency compared all but one abstractive model. We believe this is a result of minimizing the text passed through LLMs and offloading the summarization to our methods instead, which retained our original readability and Having two more mediums (the summary and reduced causal map) that are simpler to understand while minimizing information loss allows more users to understand the problem on simpler terms both structurally and textually, as a way to highlight key concepts that would otherwise be overshadowed by noise.

References

[1] K. L. Sainani, Explanatory versus predictive modeling, PM&R 6 (9) (2014) 841–844.

[2] O. Osoba, B. Kosko, Causal Modeling with Feedback Fuzzy Cognitive Maps, John Wiley & Sons, Ltd, 2019, Ch. 25, pp. 587–615. `arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119485001.ch25`, `doi:https://doi.org/10.1002/9781119485001.ch25`.
URL `https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119485001.ch25`

[3] B. Kosko, Fuzzy cognitive maps, International journal of man-machine studies 24 (1) (1986) 65–75.

[4] H. De Pinho, Generation of systems maps, Systems Science and Population Health (2017) 61–76.

[5] P. J. Giabbanelli, K. L. Rice, M. C. Galgoczy, N. Nataraj, M. M. Brown, C. R. Harper, M. D. Nguyen, R. Foy, Pathways to suicide or collections of vicious cycles? understanding the complexity of suicide through causal mapping, Social network analysis and mining 12 (1) (2022) 60.

[6] A. Jack, Foresight report on obesity–author's reply, The Lancet 370 (9601) (2007) 1755.

[7] S. Allender, B. Owen, J. Kuhlberg, J. Lowe, P. Nagorcka-Smith, J. Whelan, C. Bell, A community based systems diagram of obesity causes, PloS one 10 (7) (2015) e0129683.

[8] B. P. Rigby, C. J. Dodd-Reynolds, E. J. Oliver, The understanding, application and influence of complexity in national physical activity policy-making, Health Research Policy and Systems 20 (1) (2022) 59.

[9] W. S. El-Kassas, C. R. Salama, A. A. Rafea, H. K. Mohamed, Automatic text summarization: A comprehensive survey, Expert systems with applications 165 (2021) 113679.

[10] Y. Liu, T. Safavi, A. Dighe, D. Koutra, Graph summarization methods and applications: A survey, ACM computing surveys (CSUR) 51 (3) (2018) 1–34.

[11] P. Verma, H. Om, Mcrmr: Maximum coverage and relevancy with minimal redundancy based multi-document summarization, Expert Systems with Applications 120 (2019) 43–56.

[12] P. T. Hester, K. M. Adams, Complex systems modeling, in: Systemic Decision Making, Springer, 2017, Ch. 5, pp. 101–125.

[13] A. K. Tsadiras, Comparing the inference capabilities of binary, trivalent and sigmoid fuzzy cognitive maps, Information Sciences 178 (20) (2008) 3880–3894.

[14] P. T. Hester, M. Akpinar-Elci, J. M. Shaeffer, M. B. Shaeffer, On complementarity and the need for a transdisciplinary approach in addressing emerging global health issues, Transdisciplinary perspectives on complex systems: New findings and approaches (2017) 193–212.

[15] G. Nápoles, N. Ranković, Y. Salgueiro, On the interpretability of fuzzy cognitive maps, Knowledge-Based Systems 281 (2023) 111078.

[16] E. I. Papageorgiou, M. F. Hatwágner, A. Buruzs, L. T. Kóczy, A concept reduction approach for fuzzy cognitive map models in decision making and management, Neurocomputing 232 (2017) 16–33.

[17] K. McPherson, T. Marsh, M. Brown, Foresight report on obesity, The Lancet 370 (9601) (2007) 1755.

[18] A. Phatak, V. K. Mago, A. Agrawal, A. Inbasekaran, P. J. Giabbanelli, Narrating causal graphs with large language models, arXiv preprint arXiv:2403.07118 (2024).

[19] A. Shrestha, K. Mielke, T. A. Nguyen, P. J. Giabbanelli, Automatically explaining a model: Using deep neural networks to generate text from causal maps, in: 2022 Winter Simulation Conference (WSC), IEEE, 2022, pp. 2629–2640.

[20] C. Zhu, Chapter 8 - applications and future of machine reading comprehension, in: C. Zhu (Ed.), Machine Reading Comprehension, Elsevier, 2021, pp. 185–207. doi:https://doi.org/10.1016/B978-0-323-90118-5.00008-4.
URL https://www.sciencedirect.com/science/article/pii/B9780323901185000084

[21] L. Torrey, J. Shavlik, Transfer learning, in: Handbook of research on machine learning applications and trends: algorithms, methods, and techniques, IGI global, 2010, pp. 242–264.

[22] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, et al., Exploring the limits of transfer learning with a unified text-to-text transformer, Journal of Machine Learning Research 21 (140) (2020) 1–67.

[23] R. Mihalcea, P. Tarau, Textrank: Bringing order into text, in: Proceedings of the 2004 conference on empirical methods in natural language processing, 2004, pp. 404–411.

[24] R. Nallapati, B. Zhou, C. Gulcehre, B. Xiang, et al., Abstractive text summarization using sequence-to-sequence rnns and beyond, arXiv preprint arXiv:1602.06023 (2016).

[25] A. See, P. J. Liu, C. D. Manning, Get to the point: Summarization with pointer-generator networks, arXiv preprint arXiv:1704.04368 (2017).

[26] D. Miller, Leveraging bert for extractive text summarization on lectures, arXiv preprint arXiv:1906.04165 (2019).

[27] I. Beltagy, M. E. Peters, A. Cohan, Longformer: The long-document transformer, arXiv preprint arXiv:2004.05150 (2020).

[28] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 7871–7880.

[29] R. Řehůřek, P. Sojka, Software Framework for Topic Modelling with Large Corpora, in: Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks, ELRA, Valletta, Malta, 2010, pp. 45–50, http://is.muni.cz/publication/884893/en.

[30] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: pre-training of deep bidirectional transformers for language understanding, CoRR abs/1810.04805 (2018). arXiv:1810.04805.
URL http://arxiv.org/abs/1810.04805

[31] O. Olabisi, A. Hudson, A. Jetter, A. Agrawal, Analyzing the dialect diversity in multi-document summaries, in: Proceedings of the 29th International Conference on Computational Linguistics, International Committee on Computational Linguistics, Gyeongju,

Republic of Korea, 2022, pp. 6208–6221.
URL https://aclanthology.org/2022.coling-1.542

[32] S. Vatsal, A. Meyers, J. Ortega, Classification of us supreme court cases using bert-based techniques, arXiv preprint arXiv:2304.08649 (2023).

[33] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., Huggingface's transformers: State-of-the-art natural language processing, arXiv preprint arXiv:1910.03771 (2019).

[34] P. von Platen, Led large 16384 pubmed, https://huggingface.co/patrickvonplaten/led-large-16384-pubmed, accessed: 2023-10-30 (2023).

[35] I. Obonyo, S. Casola, H. Saggion, Exploring the limits of a base bart for multi-document summarization in the medical domain, in: Proceedings of the Third Workshop on Scholarly Document Processing, 2022, pp. 193–198.

[36] P. Manakul, M. J. Gales, Long-span summarization via local attention and content selection, arXiv preprint arXiv:2105.03801 (2021).

[37] R. Meng, K. Thaker, L. Zhang, Y. Dong, X. Yuan, T. Wang, D. He, Bringing structure into summaries: a faceted summarization dataset for long scientific documents, arXiv preprint arXiv:2106.00130 (2021).

[38] A. Gupta, D. Chugh, Anjum, R. Katarya, Automated news summarization using transformers, in: Sustainable Advanced Computing: Select Proceedings of ICSAC 2021, Springer, 2022, pp. 249–259.

[39] A. Gatt, E. Krahmer, Survey of the state of the art in natural language generation: Core tasks, applications and evaluation, Journal of Artificial Intelligence Research 61 (2018) 65–170.

[40] W. Li, W. Wu, M. Chen, J. Liu, X. Xiao, H. Wu, Faithfulness in natural language generation: A systematic survey of analysis, evaluation and optimization methods, arXiv preprint arXiv:2203.05227 (2022).

[41] Wikipedia contributors, Simon tisdall — Wikipedia, the free encyclopedia, [Online; accessed 26-March-2024] (2024).
URL https://en.wikipedia.org/w/index.php?title=Simon_Tisdall&oldid=1214072629

[42] M. Schmitt, S. Sharifzadeh, V. Tresp, H. Schutze, An unsupervised joint system for text generation from knowledge graphs and semantic parsing, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 7117–7130.